3.2.5 Finite state machine diagram

The use case diagram in figure 3.3 provides only static details; it has no way to depict the dynamic timing and state transitions that the ballot process requires. Moreover, the diagram doesn’t impose the order in which the operations should occur: registration period, voting period, and the determination of the winner. Do you now see the need for another design diagram that depicts the system dynamics?

To represent system dynamics, you’ll use a UML finite state machine, or FSM diagram. The FSM is well founded in formal computer science and mathematics, but it is also a versatile UML design diagram. It is an important diagram because it represents the various state changes a smart contract goes through that are dependent on time and other conditions. Often, the conditions and rules are based on various phases of a real-world contract or process, which brings us to design principle 5.

> **DESIGN PRINCIPLE 5** Use a finite state machine UML diagram to represent system dynamics such as state transitions within a smart contract.

In a voting process, voters are registered first, and there usually are deadlines for registration and voting. In some U.S. states, you have to be registered 30 days before the election day, and the voting takes place and is completed in a single day for in-person voters. If that is the case

- Registration has to be completed before voting and before a specific deadline.
- The functions for the ballot process proceed in a certain sequence.
- Voting is open only for a specified period.
- The winner can be determined only after the voting.